

FOL (Functional Object Lisp) demonstrates that CLOS-style object orientation and hand-coded persistent data structures are synergistic rather than competing paradigms. We present the design and implementation of FOL, showing how persistent objects and CLOS can be synthesized to yield metaobject-protocol-enabled versioned objects, declarative event sourcing via method combinations, and content-based observable notifications via predicate dispatch. These patterns are more ergonomic in FOL than in either parent language alone. Our contributions are two-fold: (1) a MOP adaptation for persistent slot storage using a hybrid native-slot/overflow-trie layout ($T=8$) and a lazy schema-evolution strategy that trades explicit migration hooks for corruption safety; and (2) a four-level category-based specificity ordering for predicate dispatch that achieves deterministic, definition-order tie-breaking while unifying dispatch across named and anonymous functions. Benchmarks show overhead ratios from $23.2\times$ for micro-benchmarks down to $1.27\times$ for a 4,096-gate digital logic simulator at scale, confirming that persistent data structure overhead amortizes as application computation grows.

FOL: A Functional Object Lisp

Frank Adrian

Ancar Technology

Olathe, KS, USA

frank.adrian314159@gmail.com

CCS Concepts

• **Software and its engineering** → **Functional languages; Object oriented languages; Extensible languages.**

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

FOL explores the synthesis of Common Lisp-style object orientation (CLOS) and Clojure-style immutable data structures. We find that multiple dispatch over persistent state enables architectural patterns that are more natural than in either parent paradigm alone. While CLOS offers powerful method combinations and a metaobject protocol (MOP), its root in mutable state complicates versioned history. Conversely, Clojure's immutable model lacks the introspective extensibility provided by the MOP.

Immutable objects simplify concurrent development by eliminating synchronization conflicts and manual locking. FOL implements these objects by adapting the CLOS MOP to use immutable data structures for slot storage. Using the MOP for persistence is not itself new—Racket provides immutable structs, and functional record types appear in F# and Elm—but applying it to full CLOS with method combinations, the slot-missing overflow protocol, and cross-metaclass inheritance raises specific engineering questions that prior work does not address: how to lay out slots efficiently across the native/overflow boundary, where to set the crossover threshold, and how to handle class redefinition without corrupting old instances. We implemented FOL as a standalone Lisp rather than a library to ensure language-level correctness; a library-based approach would struggle to enforce persistence invariants and manage the integration of persistent and primitive types.

The synthesis of immutability and CLOS makes several patterns more ergonomic than in either parent alone:

- **Versioned objects via the MOP:** Functional updates transparently produce instances of the new schema while preserving old snapshots, trading CLOS's explicit migration hooks for corruption safety.
- **Event sourcing via method combinations:** Declarative : around methods intercept all mutations to build immutable audit logs automatically, a guarantee harder to enforce in Clojure.
- **Observable notifications via predicate dispatch:** Content-based routing of change events without manual cond cascades, combining persistent updates with extensible generic functions.

Beyond architectural synergy, FOL provides concise predicate dispatch and enhanced destructuring while maintaining acceptable performance for workloads that benefit from immutability guarantees.

1 Language Design

1.1 Core Philosophy

Throughout this paper, *persistent* refers to the data structure sense of Driscoll et al. [13]—preserving previous versions after modification via structural sharing—not database persistence.

FOL makes several design commitments:

- (1) **Immutability by default:** All objects and collections (except streams, atoms, etc.) are persistent, preserving previous versions after modification.
- (2) **Structural sharing:** Updates create new versions sharing structure with old.
- (3) **Object identity:** Distinguishes version identity (eq) from value equality (=).
- (4) **Generic functions:** Multiple dispatch over destructuring patterns.
- (5) **Meta-object protocol:** Introspection and extension via adapted MOP protocols.

1.2 Identity and Equality

Following Hickey's epochal time model [19], FOL simplifies Common Lisp's four-level equality hierarchy (eq/eql/equal/equalp) to two levels. eq tests *reference identity*—each functional update produces a new object, so (eq alice older-alice) returns false even when both represent "Alice." FOL's = tests *value equality* via generic dispatch: numbers compare with numeric coercion, persistent objects compare by structural comparison of storage maps, and strings compare by content—subsuming the roles of eql, equal, and equalp. Two independently constructed objects with identical slot values are = and may be used as map keys.

1.3 Syntax and Readability

FOL adopts Clojure's literal and definition syntax for conciseness. Persistent collections use literal syntax with Clojure semantics:

```
[1 2 3]           ; Vectors
{:name "Alice" :age 30} ; Maps
#{1 2 3 4}        ; Sets
```

```
(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])
```

```
(defn summarize [{:keys [name age]]
  (str name " is " age " years old"))
```

Type names use Dylan-style angle brackets (<person>). The language also provides the bind special form for sequential lexical bindings (replacing Common Lisp's let*), as well as Clojure's threading macros (->, ->») for deeply nested calls.

1.4 Persistent Object Protocol

By default, user-defined classes inherit from `<persistent-object>`:

```
(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])

(def alice (make <person> :name "Alice" :age 30))
(def older-alice (assoc alice :age 31))
(:age alice)      ; => 30 (unchanged)
(:age older-alice) ; => 31
```

`defclass` supports standard CLOS options (`:type`, `:initarg`, `:initform`, `:reader`). Keywords in function position act as accessor functions (Clojure-style).

Slot storage uses a hybrid strategy inspired by V8’s hidden classes [6]: objects with ≤ 8 slots store all values in native CLOS slots, providing native memory access speed; objects with > 8 slots store the first 8 in native CLOS slots and overflow the remainder into a persistent vector trie. Functional updates (`assoc`) shallow-copy the native slots and, for wide objects, batch-construct a new persistent vector from the overflow slot values, keeping updates $O(k)$ for an object with $k \leq 8$ slots and $O(1) + O(\log n)$ for the overflow portion of wider objects.

The threshold of 8 is motivated by empirical class-size data and micro-benchmark results. Real-world OO class sizes are heavily right-skewed: Concas et al. [11] measure mean 4.6, median 2, and $SD \approx 8.3$ fields per class in Eclipse (Java, $n = 10,000$ classes), consistent with findings across languages [2, 23]. The 75th percentile is approximately 5–7 fields and the 90th percentile approximately 10–12, so $T = 8$ keeps roughly 80% of realistic class definitions on the all-native fast path. Micro-benchmarks over candidate thresholds $T \in \{8, 12, 16, 24, 32\}$ (Section 5.5) further show that persistent vector batch trie construction costs ≈ 106 ns base plus 5–7 ns per overflow element—substantially less per element than native slot-value dispatch (≈ 27 ns/slot). Copying 8 native slots then batch-building the overflow vector is therefore faster than copying 32 native slots individually for objects with more than ≈ 28 slots, while objects within the threshold are unaffected across all candidates.

2 Architecture and Implementation

2.1 Collection Implementation

FOL uses hand-coded persistent data structures for its collections:

- **Vectors** (`[1 2 3]`): 32-way branching persistent tries supporting $O(\log_{32} n)$ random access and $O(\log_{32} n)$ persistent updates via path copying with structural sharing.
- **Maps** (`{:a 1 :b 2}`): Hash array mapped tries (HAMTs) with $O(\log_{32} n)$ lookup, insertion, and deletion. HAMT lookup is only 1.3× slower than CL `gethash`.
- **Sets** (`#{1 2 3}`): HAMTs storing element presence, sharing the map implementation.
- **Sorted collections**: B-trees with configurable comparators for ordered maps and sets.
- **Lists**: Persistent cons cells with $O(1)$ prepend and $O(n)$ access.

Beyond these core types, FOL provides thirteen additional collection classes, from `<array-dict>` to `<dense-int-set>` to `<array>`.

All are implemented using the hand-coded HAMT, persistent vector trie, and B-tree primitives, or have been hand-written for collections not based on persistent structures (list, lazy-seq). The self-contained implementation eliminates external library dependencies for collection storage.

2.2 Runtime Representations

Most runtime representations other than collections and user-defined objects utilize a Common Lisp base. Primitives (`<bool>`, `<string>`, the numeric tower, `<symbol>`) are unwrapped CL values; functions with their closures are CL functions; the package system and error handling derive from CL. Garbage collection is inherited—old versions of user-defined objects are collected when unreferenced; structural sharing ensures only unshared portions are reclaimed.

2.3 MOP Protocol Adaptation

FOL’s persistent metaclass adapts rather than replaces the MOP. Table 1 summarizes the protocol disposition.

Introspection protocols (`class-slots`, `class-precedence-list`, `slot-definition-*`) work unchanged because FOL’s class *structure* is standard CLOS—only slot *mutation* is guarded. For objects with ≤ 8 slots, `slot-value-using-class` uses standard CLOS slot reads at native speed; the `setf` method guards against post-initialization writes. For wide objects (> 8 slots), overflow slots are intercepted by `slot-missing` and redirected to a persistent vector trie. The `validate-superclass` extension permits cross-metaclass inheritance (persistent classes may inherit from standard classes). This adaptation requires the MOP: persistence cannot be implemented as a library because transparent mutation guarding and overflow routing demand metaclass-level interception.

The contribution of this adaptation is not the concept of metaclass-level immutability per se, but three concrete engineering decisions: (1) the hybrid native-slot/overflow-trie layout that preserves $O(1)$ access for the common case; (2) the threshold $T = 8$ chosen by cross-referencing empirical class-size data with micro-benchmark crossover analysis (Section 5.5); and (3) the lazy migration strategy for class redefinition (Section 4.4), which arises naturally from the combination of functional updates and CLOS introspection and has not, to our knowledge, been described for immutable object systems.

3 Features

3.1 Clojure Features

FOL implements key Clojure abstractions:

- **Transducers** [20]: Composable transformation pipelines separating the “what” (map, filter, take) from the “how” (eager, lazy, parallel).
- **Lazy sequences**: On-demand computation enabling infinite sequences and efficient streaming.
- **Atoms, Refs/STM, and Agents**: Clojure’s concurrency primitives for managing mutable state in a thread-safe manner.
- **Sequence functions**: Over 100 functions including map, filter, reduce, take, drop, partition, group-by, and frequencies.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
slot-value-using-class	Standard	Native CLOS slot read
slot-missing	Extended	Overflow to persistent vector trie
(setf slot-value-using-class)	Guarded	Blocks post-initialization writes
slot-makunbound-using-class	Blocked	Immutability invariant
validate-superclass	Extended	Cross-metaclass mixing
initialize-instance	Extended	Populates map from initargs
update-instance-for-redefined-class	Omitted	Lazy migration (Sec. 5.4)
change-class	Omitted	Violates immutability

- **Macros:** Clojure-style defmacro with syntax-quote, unquote, splicing unquote, and auto-gensym.

```
[[[x (<vector>?)] (vec (map process-value x))]]
[[x] x)]
```

3.2 Generic Function Integration

FOL’s generic functions dispatch by arity first, then by type predicates and specificity within an arity group. Methods at different arities may coexist on the same generic:

```
(defgeneric process [x])
(defmethod process
  [[(x <vector>)] (vec (map process x)))]
  [[(x <dict>)] (update-vals x process)]
  [[(x <number>)] (* x 2)])

(process [1 {:a 2 :b 3} [[4]]]) ; => [2 {:a 4 :b 6} [[8]]]
```

Multi-clause defmethod forms expand to separate method definitions. The `:before`, `:after`, and `:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

defn vs defgeneric/defmethod: `defn` defines a standalone multi-clause function with predicate dispatch and specificity ordering; it does not create a generic function. `defgeneric` declares a generic function; `defmethod` adds methods to it. Only `defmethod` accepts qualifiers. Both `defn` and `defmethod` support multi-clause predicate dispatch.

Figure 1 gives the formal grammar for FOL’s multi-pattern dispatch syntax.

Note that *pattern* forms defined in the grammar lack a body (used for `defgeneric` declarations), while *clause* forms combine a *pattern* and evaluating *body* (used for implementations).

3.3 Predicate Dispatch

FOL extends pattern matching with predicate specializers. The syntax `(var (fn arg0 arg1 ...))` evaluates `(fn var arg0 arg1 ...)` at runtime—the bound value is inserted as the first argument. Predicates work with any function, macro, or special form—equality, comparison, type tests, or user-defined:

```
;; Range checking with comparison predicates
(defn age-group
  [[(age (< 13))] :child]
  [[(age (< 20))] :teen]
  [[(age (< 65))] :adult]
  [[age] :senior))

;; Type predicates
(defn process-value
  [[(x (<number>?))] (* x 2)]
  [[(x (<string>?))] (str x x)])
```

3.3.1 Predicate Specificity. FOL orders patterns by *constraint strength*—how narrowly a pattern constrains its accepted values. More specific patterns are tested first, ensuring predictable dispatch regardless of definition order:

Table 2: Predicate Specificity Levels

Level	Pattern Type	Constraint	Example
3	Predicate	Single value/narrow range	<code>(n (= 5))</code>
2	Type	All instances of a type	<code>(x <number>)</code>
1	Destructuring	Structural shape only	<code>[a b c]</code>
0	Any	Universal match	<code>x, _</code>

The design space for ordering clauses with arbitrary predicate specializers contains three well-studied options, each with a disqualifying cost for FOL’s goals.

Logical subsumption (Ernst et al. [15]; also the basis of Cecil [9]) orders clauses by implication: $c_1 \prec c_2$ if every input accepted by c_1 is also accepted by c_2 . This achieves a semantically clean, programmer-intent-driven ordering, but requires deciding the subsumption relation between arbitrary predicates—a problem undecidable in general. In practice, Cecil requires explicit `implies` annotations; FOL rejects this because it imposes a verification burden that conflicts with the language’s interactive, incremental character.

Explicit priorities (Filtered Dispatch [25]) attach a programmer-assigned numeric priority to each clause. This is decidable and simple but is brittle in a multi-clause dispatch form: adding a new clause anywhere in the file can silently shift the effective order of existing clauses if the programmer misassigns its priority, and priorities assigned in isolation by different authors compose poorly across module boundaries.

Category-based specificity (this work) partitions specializers into four syntactically recognizable constraint levels (Table 2) and sorts by level, with definition order as the tie-breaker within a level. This is decidable by inspection, requires no annotations, and is stable under addition of new clauses: a new clause at a given level never perturbs the relative order of existing clauses at the same level. The four categories correspond to a natural intuition—predicates select narrower ranges than types, types select narrower ranges than structural shapes, shapes select narrower ranges than

<i>defn</i>	::=	(defn <i>name</i> (<i>single-clause</i> <i>multi-clause</i>))	
<i>defgeneric</i>	::=	(defgeneric <i>name</i> (<i>single-pattern</i> <i>multi-pattern</i>))	
<i>defmethod</i>	::=	(defmethod <i>name</i> <i>qualifier</i> [?] (<i>single-clause</i> <i>multi-clause</i>))	
<i>fn</i>	::=	(fn (<i>single-clause</i> <i>multi-clause</i>))	
<i>single-clause</i>	::=	<i>pattern</i> <i>body</i> ⁺	
<i>multi-clause</i>	::=	(<i>pattern</i> <i>body</i> ⁺) ⁺	
<i>single-pattern</i>	::=	<i>pattern</i>	
<i>multi-pattern</i>	::=	<i>pattern</i> ⁺	
<i>pattern</i>	::=	[<i>param</i> [*]] [<i>param</i> [*] & <i>rest-param</i>]	
<i>param</i>	::=	<i>symbol</i>	(simple binding)
		(<i>symbol</i> <i>type</i>)	(type specifier)
		(<i>symbol</i> (<i>fn-name</i> <i>arg</i> [*]))	(predicate specifier)
		<i>destructure</i>	(nested destructuring)
<i>destructure</i>	::=	[<i>param</i> [*]]	(sequential)
		{ :keys [<i>key-spec</i> [*]] }	(map)
<i>key-spec</i>	::=	<i>symbol</i> (<i>symbol</i> <i>type</i>) (<i>symbol</i> <i>pred</i>)	
<i>type</i>	::=	< <i>type-name</i> >	
<i>qualifier</i>	::=	:before :after :around	

Figure 1: FOL Multi-Pattern Dispatch Grammar (simplified)

wildcards—making the ordering predictable without requiring the programmer to reason about subsumption.

Within a specificity level, definition order is preserved, giving programmers explicit control over tie-breaking. For sequence patterns, specificity extends to element-level comparisons: $[(x \text{ < } \emptyset)]$ y is more specific than $[(x \text{ <number>}) y]$ because its first element has higher specificity. In cases like $[(x (= 5)) (y \text{ <number>})]$ versus $[(x \text{ <number>}) (y (= 5))]$, the element-wise comparison rules ensure deterministic ordering without ambiguity. Because $(= 5)$ is a Level 3 predicate and <number> is a Level 2 type constraint, the algorithm dictates that the $(= 5)$ check in the first position (Spec-Prefix) strictly supersedes the lower-level type check.

3.3.2 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 3. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p .

Table 3: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Prefix	$\frac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \quad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\frac{\mathcal{L}(p_i) = \mathcal{L}(q_i) \ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n \ \& \ r]}$

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $\text{idx}(p)$ denote the definition index of the method clause

containing p .

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge \text{idx}(p_1) < \text{idx}(p_2))$$

Patterns of different arity are incomparable under $\prec$; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$ within it. This lexicographic combination ensures deterministic dispatch: arity-first, then specificity, then definition-order.

3.3.3 Dispatch Semantics. Predicate specializers work in *defn*, *defmethod*, and anonymous *fn* forms but not in macro parameter lists (*defmacro* rejects type and predicate specializers). Unlike standard CLOS—which memoizes effective methods for $O(1)$ amortized dispatch based purely on class precedence signatures—FOL’s predicate dispatch requires an $O(N)$ linear scan on every invocation, where N is the number of patterns at the given arity. Because predicates are arbitrary functions whose return values may change based on dynamic state, resolution cannot be statically cached. While methods are pre-sorted by specificity at definition time to minimize evaluation of weaker constraints, the linear $O(N)$ evaluation cost means predicate dispatch is fundamentally a high-level routing mechanism rather than a low-level, tight-loop optimization. FOL’s type system is entirely dynamic—type specializers are checked at runtime. The boundary between type and predicate categories is conventional: type predicates like <number>? are recognized syntactically and assigned Level 2, consistent with type specializers. This resolves the correctness hazard of silently incorrect dispatch when mixing types and type predicates. While the language permits mixing, using type specializers is encouraged for clarity.

FOL’s dispatch maintains three invariants: (1) **Exhaustive matching**—unmatched calls signal `no-matching-clause` rather than silently failing; (2) **Deterministic dispatch**—the total order $\prec_{\text{disp}}$ ensures exactly one clause matches for any input; (3) **Type predicate consistency**—type specializers use the same predicates as explicit checks, so $(x \text{ <number>})$ matches exactly the values for which $(\text{<number>? } x)$ returns true.

4 Synergy in Practice

This section demonstrates patterns that are *more natural or more concise* in FOL than in either Clojure or standard CLOS alone, because they exploit the combination of persistent data, CLOS method combinations, and predicate dispatch. Full source for all examples is available in the repository: FOL code in `fol-code/`, Common Lisp equivalents in `lisp-code/`.

4.1 Invariant Guards

FOL enables structural integrity via predicate-specialized method combinations. A common requirement is to enforce cross-cutting invariants (e.g., account balance ≥ 0) across all mutation paths. Without method combinations, this guard must be duplicated manually in every function that modifies the object:

```
(defn deposit [acc amount]
  (check-balance (- (balance acc) amount)) ...)
(defn withdraw [acc amount]
  (check-balance (- (balance acc) amount)) ...)
```

In FOL, the invariant is declared once as an `:around` method on `assoc`, firing automatically regardless of which function initiates the change:

```
(defmethod assoc :around [(acc <account>) slot val]
  (bind [result (call-next-method)]
    (when (< (:balance result) 0)
      (error "~A: balance cannot go negative" (:owner acc)))
    result))
```

Because FOL objects are immutable, the original snapshot remains valid if an error is signaled; rollback logic is structurally unnecessary. This demonstrates the synergy: persistent objects provide the safe snapshot, while CLOS method combinations provide the declarative, cross-cutting interceptor.

4.2 Derived-Value Invalidation

Derived-value caching (e.g., total price of a shopping cart) often suffers from stale-cache bugs when upstream dependencies change. FOL solves this by specializing an `:around` method on `assoc` for the specific upstream slot:

```
(defmethod assoc :around [(cart <cart>) (slot (= :items)) val]
  (assoc (call-next-method) :_total nil))
```

The invalidation fires only when `:items` is modified, leaving unrelated updates unaffected. The lazy accessor recomputes the sum on the next read and stores it back to warm the cache. This pattern synergistically combines predicate-dispatch (matching the `(= :items)` slot keyword) with the versioned nature of persistent objects, ensuring that no mutation path can accidentally produce a stale derived value.

4.3 Observable Slots via MOP and Predicate Dispatch

This example combines persistent functional updates with predicate dispatch to build a change-notification system. When a slot is updated, the protocol produces both a new object and a change event; predicate dispatch routes notifications by content (e.g., detecting reading spikes or critical status transitions):

```
(defclass <sensor> []
  [[name :initarg :name]
```

```
  [reading :initarg :reading :initform 0]
  [status :initarg :status :initform :normal]])

(defn updated [obj slot new-val]
  (bind [old-val (get obj slot)
        new-obj (assoc obj slot new-val)]
    {:object new-obj
     :change {object obj :slot slot
              :old old-val :new new-val}}))

(defn spike? [ch]
  (bind [change (get ch :change)]
    (and (= (get change :slot) :reading)
         (> (- (get change :new)
              (get change :old)) 50))))

(defgeneric on-change [change])
(defmethod on-change
  ([[(c (spike?))]
   {alert :spike
    :message (str "Spike detected: "
                  (get (get c :change) :old) " -> "
                  (get (get c :change) :new))}]
   [(c (critical?))]
   {alert :critical
    :message "Sensor entered critical state"})]
  [c]
  {alert :info
   :message (str "Slot " (get (get c :change) :slot)
                 " changed"))}))
```

This example illustrates three-way synergy: persistence provides safe functional updates that produce change events without corrupting the original; predicate dispatch routes those events by content; and generic functions with `defmethod` enable new notification rules to be added without modifying the existing dispatch. None of these features alone suffices—Clojure has persistence but no method dispatch for extensible routing; CLOS has dispatch but requires manual deep copies to produce safe change events.

4.4 Lazy Schema Evolution

Standard CLOS handles class redefinition by lazily updating existing instances via `update-instance-for-redefined-class`: each instance is mutated in place the next time it is accessed. This protocol assumes mutable instances—an assumption FOL deliberately violates.

FOL's persistent metaclass resolves this tension through what amounts to *lazy schema evolution*. Class definitions remain mutable (redefining a class updates the class object normally), but existing instances are frozen snapshots whose native CLOS slots and overflow vectors are never modified after initialization. The MOP provides graceful degradation: when accessing a slot added by redefinition but absent from an old instance, the system falls back to the slot's `:initform` if one exists, or signals `slot-unbound` otherwise. Slots removed by redefinition remain in the instance but become inaccessible (no slot definition routes to them).

Crucially, functional updates perform implicit migration. `assoc` calls `(class-of object)` to obtain the *current* (redefined) class, allocates a fresh instance of that class, and shallow-copies the old slots forward. The resulting object is an instance of the new schema carrying the old data. Listing 1 demonstrates this: an instance `v1`

Listing 1: Lazy Schema Evolution

```
(defclass <user> [] [[id] [login]])
(bind [v1 (-> (make <user>) (assoc :id 1) (assoc :login "bob"))])

;; Redefine class: add :email, remove :login
(defclass <user> [] [[id] [email :initform "none"]])

(bind [v2 (assoc v1 :id 2)]) ;; Implicit migration
(get v1 :login) ;; => "bob" (v1 is still Schema 1)
(get v2 :login) ;; => unbound or missing (v2 is Schema 2)
(get v2 :email) ;; => "none" (v2 has Schema 2 defaults)
```

created under the original class definition coexists with its successor v2, which has transitioned to the new schema.

This lazy migration strategy trades explicit transformation control for absolute corruption safety. However, it can produce surprising behavior if the programmer renames a slot: `assoc` on the old instance will lose the data under the old name, as the new class has no record of it. This is a form of graceful degradation that ensures the system never crashes due to schema drift, but it requires that persistent history be navigated with awareness of the schema version. It is, to our knowledge, a novel adaptation of the MOP for immutable object systems.

5 Evaluation

5.1 Comparison with Related Languages

Table 4: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Persistent objects	✓	✓	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single ^a	×
Transducers	✓	✓	×
Lazy seqs	✓	✓	×
Macros	✓	✓	✓

^a `defmulti` provides single-parameter predicate dispatch (arbitrary fn) but lacks multi-parameter specificity ordering.

^b Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.

FOL uniquely combines Clojure’s functional features with Common Lisp’s object system. Persistent objects with fewer than 8 slots maintain $O(1)$ access and mutable-equivalent update cost. Those with more than 8 slots and most collections incur $O(\log n)$ vs. $O(1)$ for mutable operations, but high branching factors (32–64) keep constant factors small.

5.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Feature benchmarks report the mean of 100 runs (coefficient of variation <2%); LSim benchmarks report the mean of 10 runs. We compare against native Common Lisp rather than Clojure: SBCL compiles to native code, providing a strict upper bound on overhead, and since FOL transpiles to Common

Lisp, this isolates abstraction cost. A Clojure comparison would conflate language design with JVM runtime differences.

5.3 Feature Benchmarks

We evaluated FOL across three benchmarks exercising specific language features: invariant guards, derived-value invalidation, and observable slots. Table 5 summarizes performance and memory ratios.

Table 5: Feature Benchmark Summary

Benchmark	Perf Ratio	Mem Ratio
Invariant Guards	23.2×	8.0×
Derived-Value Invalidation	19.7×	948×
Observable Slots	6.69×	6.32×

Invariant Guards: This benchmark measures the happy-path cost of an `:around` method that validates a balance invariant on every `assoc`. The 23.2× overhead stems from generic dispatch and the `:around` method trampoline, with 8.0× memory pressure from structural sharing path-copying.

Derived-Value Invalidation: Implements automatic cache clearing for dependent slots. It shows 19.7× time overhead and 948× memory allocation. The extreme memory ratio reflects both HAMT path-copying across three internal `assoc` calls and the allocation of persistent vector nodes for a growing dependency list.

Observable Slots: Implements a monitoring loop with change-event routing. The 6.69× overhead reflects the cost of producing persistent maps for both the updated state and the event record, with 6.32× memory overhead.

The feature benchmarks show overhead ratios from 6.7× to 23.2×, depending on the number of persistent updates per logical operation. These ratios represent the fundamental cost of FOL’s persistent object model where correctness and structural integrity are prioritized over raw throughput. Syntactic abstraction overhead is minimized by the compiler, which emits direct `defun` calls for internal function definitions. Remaining non-object overhead stems from closure allocation and runtime dispatch checks for late-bound or external symbols (where a call must support fallback to collection access).

5.4 Logic Simulation (LSim)

LSim is a digital logic simulator that exercises persistent state management at scale. We evaluated four configurations of increasing complexity: 8bit-100 (small), 32bit-300 (medium), 8x32-900 (a 1024-gate pipeline of eight cascaded 32-bit registers), and 32x32-3000 (a 4096-gate pipeline of thirty-two cascaded 32-bit registers with 3000 time steps). Table 6 presents the results.

The performance ratio narrows from 7.0× to 1.27× as circuit complexity grows. This trend reflects amortization rather than elimination of overhead: the absolute cost of persistent allocation remains, but it becomes a smaller fraction of total runtime as the simulation computation grows. At the largest scale (32x32-3000, ~4096 NAND gates, 3000 time steps), simulation work dominates and the persistent data structure overhead accounts for only 27% of total execution time. Crucially, in contexts like simulation where

Table 6: LSim Scaling Results

Config	CL	FOL	Ratio
8bit-100	3 ms	22 ms	7.0×
32bit-300	14 ms	170 ms	12.1×
8x32-900	1.02 s	4.26 s	4.2×
32x32-3000	48.3 s	61.3 s	1.27×

state history must be preserved for analysis or event-replay, a mutable CLOS implementation must defensively deep-copy the entire circuit state at each time step (an $O(N)$ allocation). FOL’s structural sharing allocates only $O(\log N)$ memory for the changed branches. At scale, this structural sharing inverts the naive expectation: GC pressure benchmarks for the 32x32-3000 circuit show that FOL allocates only 0.59× the bytes of the Common Lisp version and triggers 0.59× as many GC invocations. This empirical result confirms that at the limits of state scaling, the efficiency of structural sharing outweighs the overhead of persistent allocation.

5.5 Persistent Object Micro-Benchmarks

To isolate the cost of FOL’s persistent metaclass from higher-level transpiler overhead, we measured construction, read, and functional update costs for persistent objects versus standard mutable CLOS objects across a range of slot counts. Classes were generated dynamically with identical slot definitions; persistent classes use persistent-class as metaclass and inherit from <persistent-object>, while mutable classes use standard standard-class. Overflow storage (> 8 slots) uses a persistent vector trie. Tables 7 and 8 were collected at the pre-update threshold of $T=32$ and compare persistent against mutable storage; Table 9 presents the threshold selection analysis.

Table 7: Persistent Object Construction Overhead

Slots	Memory (B/obj)			Time		
	Pers.	Mut.	Ratio	Pers.	Mut.	Ratio
2	360	216	1.67×	2.15 μ s	1.16 μ s	1.85×
8	1,277	747	1.71×	3.79 μ s	4.79 μ s	0.79×
32	4,924	2,857	1.72×	13.8 μ s	10.8 μ s	1.27×
48	7,863	4,267	1.84×	23.1 μ s	19.6 μ s	1.18×
128	22,305	11,301	1.97×	85.6 μ s	82.6 μ s	1.04×

Table 8: Persistent Object Read and Update Overhead

Slots	Slot	Read (P/M)	Ratio	Update (P/M)	Ratio
2	0	46 ns / 33 ns	1.38×	1.14 μ s / 35 ns	33×
8	0	65 ns / 31 ns	2.11×	2.34 μ s / 42 ns	56×
32	0	52 ns / 30 ns	1.73×	6.85 μ s / 29 ns	240×
48	0	55 ns / 30 ns	1.84×	6.96 μ s / 28 ns	249×
48	33	131 ns / 43 ns	3.04×	8.50 μ s / 42 ns	204×
128	0	61 ns / 33 ns	1.84×	6.78 μ s / 42 ns	161×
128	33	169 ns / 35 ns	4.85×	14.4 μ s / 32 ns	447×

Three findings stand out. First, **construction overhead is modest**: 1.67–1.72× memory and 0.79–1.85× time for objects with ≤ 8 native slots. The persistent vector trie used for overflow storage keeps memory overhead under 2× even at 128 slots (1.97×). Second, **native slot reads are fast**: persistent objects read at 1.4–2.1× the speed of mutable CLOS for slots in the native range (≤ 8). Overflow slot reads (slot 9+, intercepted via slot-missing) cost 3–5× due to the MOP intercept and persistent vector trie lookup. Third, **functional updates are the dominant cost**: update-slot is 33–447× slower than in-place (setf slot-value), because each update allocates a new instance and shallow-copies all native slots. This cost grows with slot count but is amortized in practice by batching via update-slots. The absolute cost of a functional update (1–14 μ s) is comparable to a small number of hash-map insertions, consistent with the feature benchmark overhead ratios observed in Section 5.3.

A separate micro-benchmark evaluated the five candidate thresholds by directly simulating hybrid copy at each T across object sizes $N \in \{8, 12, 16, 24, 32, 48\}$ (Table 9). Persistent vector batch trie construction costs ≈ 106 ns base plus 5–7 ns per element—an order of magnitude less per element than slot-value dispatch (≈ 27 ns/slot). For objects at or below the threshold (all-native path), all candidates are equivalent. The crossover occurs near $N \approx 28$: above this size, $T=8$ minimizes update cost. Objects in the 9–24 slot range incur a modest penalty at $T=8$ (0.12–0.17 μ s, roughly 15–21%) from applying the vector base cost to small overflows. However, empirical OO class-size studies report the 90th percentile at ≈ 10 –12 fields (median 2) [11, 23], placing $\leq 10\%$ of classes in this penalty zone while $\sim 80\%$ remain all-native. Note that the simulated costs in Table 9 represent raw copy work; the actual update-slots costs (Table 8) are 1.6–3× higher due to MOP slot iteration, slot-bound checks, and keyword-to-slot-name lookup overhead.

Table 9: Modeled update cost (μ s/op) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	0.42	0.42	0.43	0.42	0.42	all-native
12	0.75	0.62	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	0.77	8N + 8V
24	1.15	1.23	1.28	1.15	1.15	8N + 16V
32	1.39	1.47	1.51	1.58	1.52	8N + 24V
48	1.95	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. “ $kN + jV$ ” = copy k native slots then batch-build j -element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual update-slots is 1.6–3× higher due to MOP iteration overhead (see Table 8).

5.6 Adversarial Workloads

The 33–447× overhead of functional slot updates (Table 8) defines FOL’s operating boundary. While workloads that interleave updates with substantial computation (like LSim) amortize this cost effectively, FOL is structurally unsuitable for algorithmic workloads dominated by tight mutation loops.

Consider an adversarial benchmark such as an in-place array sort (e.g., Quicksort) or an iterative graph traversal (e.g., BFS) where localized state is mutated continuously. In native Common Lisp,

Listing 2: Naive Persistent Partition

```

929 (defn partition [v low high]
930   (bind [pivot (get v high)]
931     (loop [j low i (dec low) curr-v v]
932       (if (< j high)
933         (if (<= (get curr-v j) pivot)
934           (bind [next-i (inc i)]
935             v1 (assoc curr-v next-i (get curr-v j))
936             v2 (assoc v1 j (get curr-v next-i)))
937         (recur (inc j) next-i v2))
938       (recur (inc j) i curr-v))
939     (bind [next-i (inc i)]
940       v1 (assoc curr-v next-i (get curr-v high))
941       v2 (assoc v1 high (get curr-v next-i)))
942     [v2 next-i])))
943
944
945

```

Listing 3: Naive Persistent BFS Step

```

945 (defn bfs-step [q dists graph]
946   (if (empty? q)
947     dists
948     (bind [u (first q)]
949       edges (get graph u)
950       d (get dists u)]
951     (loop [es edges nq (rest q) nd dists]
952       (if (empty? es)
953         (bfs-step nq nd graph)
954         (bind [v (first es)]
955           (if (= (get nd v) -1)
956             (recur (rest es)
957                   (conj nq v)
958                   (assoc nd v (inc d))))
959           (recur (rest es) nq nd))))))
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980

```

these algorithms map to $O(1)$ `setf` updates against a contiguous array. In FOL, each `assoc` allocates a new instance and shallow-copies its slots, recreating the path to the root. While an idiomatic FOL developer would use a persistent tree-sort or a queue-based BFS to avoid this, we evaluated naive persistent translations of these algorithms to isolate the path-copying penalty. Listing 2 shows the naive partition logic, where every swap in the tight loop (lines 7–10) incurs two $O(\log n)$ updates and generates multiple intermediate nodes.

For a 100,000-element quicksort, this naive FOL implementation takes 885 ms (126× slower than native CL’s 7 ms) and generates 1.3 GB of garbage. The functional penalty is similarly extreme for graph traversal: a naive 50,000-node persistent BFS requires 4.2 seconds (840× slower than native CL’s 5 ms) and generates 16 GB of garbage path-copying its distance array. This empirical data highlights the exact tradeoff boundary: FOL is designed for domains that derive architectural value from immutable snapshots; for low-level, in-place algorithmic mutations, native mutable structures must be retained.

5.7 Threats to Validity

Several threats to validity should be considered. **Single implementation:** All measurements use SBCL 2.6.0 on a single platform (AMD Ryzen 9, Windows 11); results may differ on other CL implementations or architectures. **Benchmark selection:** Our benchmarks emphasize workloads that benefit from immutability (event

sourcing, simulation state management); mutation-heavy workloads (e.g., in-place graph algorithms) would show worse overhead ratios. **SLOC as a metric:** Lines of code measure syntactic density, not readability, maintainability, or correctness; shorter code is not inherently better. **Warmup effects:** Feature benchmarks report means of 100 runs, which amortizes SBCL compilation and cache warmup; the first-run cost is higher. As a single-author project, FOL has not yet undergone independent code review; the full source is publicly available to facilitate external evaluation.

6 Limitations and Future Work

FOL is transpiled to Common Lisp. No static analysis exists—type errors and unreachable clauses are detected at runtime only; some type-dependent code may require runtime resolution, degrading transpiled performance. The compiler has not been extended to warn about shadowed clauses; a future version could detect shadowing statically.

The dominant performance limitation remains the functional update cost (Section 5.6). Reducing this overhead—for example, through Clojure-style transient builders that batch mutations by briefly unguarding (`setf slot-value-using-class`) for local mutation before producing an immutable result—is a core focus of ongoing development, with initial support for object-level transient mutation now undergoing validation. Class redefinition does not migrate existing instances (Section 5.4); old objects retain their original slot data, with new slots resolved lazily via `initforms`. Our functional update pattern achieves the goal of **lenses** in the Haskell tradition, though FOL does not yet provide a formal lens abstraction.

Planned extensions include **abstract and sealed classes**, **parallel collections** following Scala’s model, **channel-based concurrency** via CSP-style go blocks, a **native compiler** to eliminate transpilation overhead, and a **VSCode extension** for integrated development support.

7 Related Work

Clojure [18] pioneered persistent data structures in Lisp. Clojure’s *protocols* provide type-based polymorphism (single dispatch on the first argument’s type) as a lightweight alternative to CLOS. FOL adopts Clojure’s sequence abstraction and transducers but uses CLOS generic functions with multiple dispatch and method combinations—gaining `:before/after/around` qualifiers and MOP introspection at the cost of protocol performance optimizations. Clojure’s `defrecord` compiles to JVM classes with direct field access and inline protocol methods, achieving near-Java performance for field reads. FOL’s `defclass` provides comparable read speed for native-range slots (≤ 8) via direct CLOS slot access, but trades Clojure’s JVM-optimized field layout for MOP extensibility—enabling metaclass-level interception that `defrecord` cannot support.

Common Lisp’s CLOS [4] and MOP [21] provide FOL’s object system foundation. While several Common Lisp libraries provide persistent data structures—notably **FSet** [7] and **immutable-structures** [26]—they typically operate as standalone collection libraries rather than integrated object systems. Metaclass-based approaches to persistence have been explored in the context of persistent object databases like **Elephant** [16], but these focus

on disk-backed storage and transactionality rather than the structural sharing and versioning of in-memory immutable data. To our knowledge, FOL is unique in providing a CLOS metaclass that transparently routes standard slot access to in-memory persistent storage with a hybrid native/overflow layout. **Dylan** [1] was an early completely object-oriented Lisp-1 and one of the early inspirations of FOL, which retains its `<type>` naming convention and uses `bind` as a replacement for `let*` as FOL’s primary lexical binding form. **Persistent data structures** build on Okasaki [24]. Scala case classes and Kotlin data classes provide value-oriented objects, but none integrate with a MOP.

7.1 Predicate Dispatch and Pattern Matching

Ernst et al. [15] introduced predicate dispatch with logical implication for specificity, building on **Cecil** [8]. Cecil’s dispatch is powerful but requires deciding the implication relation between arbitrary predicates to determine specificity—a problem that is undecidable in general and requires explicit programmer annotations in Cecil. **Multi-Java** [10] later brought predicate dispatch to a mainstream language but relied on a more restricted set of predicates to maintain decidability. FOL’s category-based specificity (Table 2) simplifies this by partitioning specializers into syntactically recognizable levels; while this lacks the mathematical generality of Cecil’s implication, it subsumes the common case for type and range tests while remaining decidable by inspection and stable under module composition. Chambers and Chen [9] demonstrated efficient implementation strategies using dispatch DAGs; **Filtered Dispatch** [25] extended CLOS with predicate-based method selection using explicit priorities. **Scala’s extractors** [14] and **Racket’s match expanders** [31] provide extensible pattern matching; FOL differs in that predicates are integrated into CLOS method combinations and specificity is determined by category level.

7.2 Functional Updates and Record Types

FOL’s functional update syntax (`update-slots`) mirrors the record update syntax found in **F#** (`{ r with A = b }`) and **Elm** [12], bringing ergonomic shallow-copy semantics with structural sharing into the Lisp object-system ecosystem. While **Haskell** utilizes mature **lenses** [22] (first-class getters and setters) for deep functional updates, FOL relies on simple threading macros like `->` operating over shallow updates, trading the mathematical compositionality of lenses for CLOS MOP-native implementation simplicity.

7.3 Multiple Dispatch and Typed Functional Lisps

Julia [3] uses multiple dispatch but is purely type-based, lacking predicate specializers, method combinations, and MOP introspection. **Coaltion** [27] adds Hindley-Milner types to CL—complementary to FOL (static safety vs. persistence). **Typed Racket** [30] adds gradual typing; FOL’s type specializers serve as dispatch mechanisms rather than correctness guarantees. **Shen** [29] and **Carp** [28] offer pattern matching and static typing respectively, but lack multiple dispatch with method combinations.

Racket [?] provides a sophisticated synthesis of functional and object-oriented features. It supports immutable records via `struct`, functional updates via `struct-copy`, and an extensible pattern

matcher via `match` [31]. While Racket’s class system [17] supports mixins and multiple interface inheritance, it remains distinct from its persistent record system. FOL differs by unifying these capabilities: a FOL object is both a CLOS instance (supporting method combinations and MOP introspection) and a persistent record (supporting `assoc` and pattern matching). This unification allows, for example, an `around` method to intercept persistent updates to a record, a pattern that requires manual coordination between classes and `structs` in Racket.

7.4 Schema Evolution and Live Systems

FOL’s lazy migration strategy (Section 5.4) shares goals with the dynamic evolution mechanisms of live object systems like **Self** [32] and **Newspeak** [5]. In **Self**, the absence of classes enables direct prototype mutation, while **Newspeak**’s first-class classes and late-bound references facilitate system-wide evolution. However, both typically assume mutable state. FOL’s contribution is showing how the combination of immutability and functional updates transforms class redefinition from a mutation that must propagate system-wide into a safety-preserving lazy migration: old instances remain valid snapshots of the previous schema, with the “migration” occurring only when needed by a functional update.

8 Conclusion

FOL demonstrates a successful synthesis of persistent data structures and CLOS, yielding architectural patterns like versioned objects and declarative event sourcing that are difficult to achieve in either paradigm isolation. Our evaluation confirms that while persistent allocation introduces significant micro-overhead (up to 23.2×), this cost is effectively amortized in non-trivial workloads, reaching near-parity (1.27×) in large-scale simulations. Crucially, we find that the efficiency of structural sharing actually inverts the naive expectation of GC pressure at scale, with FOL allocating 0.59× the bytes of a deep-copying mutable equivalent.

The primary tradeoff remains the lack of static verification for predicate dispatch, which we mitigate through a deterministic, category-based specificity ordering. This work shows that the “overhead of persistence” is a modular cost that can be managed through principled MOP-level engineering, making functional object orientation a viable and expressive choice for modern Lisp systems. The complete implementation is available at: <https://github.com/frankadrian314159/fol>

References

- [1] Inc. Apple Computer. 1992. *Dylan - An Object-Oriented Dynamic Programming Language*. Apple Computer, Inc.
- [2] Gareth Baxter, Marcus Frean, James Noble, Mark Rickerby, Hayden Smith, Matt Visser, Hayden Melton, and Ewan Tempero. 2006. Understanding the Shape of Java Software. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. 397–412.
- [3] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.
- [4] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.
- [5] Gilad Bracha, Peter von der Ahé, Vassili Bykov, Yaron Kishai, William Maddox, and David Ungar. 2010. The Newspeak programming language. In *Proceedings of the 19th symposium on Dynamic languages*.
- [6] Camillo Bruni. 2017. Fast Properties in V8. V8 Blog. <https://v8.dev/blog/fast-properties>

- [7] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. <https://common-lisp.net/project/fset/>
- [8] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 243–257.
- [9] Craig Chambers and Weimin Chen. 1999. Efficient Multiple and Predicate Dispatching. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 238–255.
- [10] Curtis Clifton, Gary T. Leavens, Craig Chambers, and Todd Millstein. 2000. MultiJava: Modular Open Classes and Symmetric Multiple Dispatch for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 130–145.
- [11] Giulio Concas, Michele Marchesi, Sandro Pinna, and Nicola Serra. 2007. Power Laws in a Large Object-Oriented Software System. *IEEE Transactions on Software Engineering* 33, 10 (2007), 687–708.
- [12] Evan Czaplicki. 2012. *Elm: Concurrent FRP for Functional GUIs*. Master’s thesis. Harvard University.
- [13] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.
- [14] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007—Object-Oriented Programming*. Springer, 273–298.
- [15] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP’98—Object-Oriented Programming*. Springer, 186–211.
- [16] Ian Eslick and Ben Hyde. 2008. Elephant: A Persistent Object Database for Common Lisp. <https://common-lisp.net/project/elephant/>
- [17] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.
- [18] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.
- [19] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. [https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_](https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md)
- Rich/AreWeThereYet.md
- [20] Rich Hickey. 2014. Transducers. Clojure Blog. <https://clojure.org/reference/transducers>
- [21] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
- [22] Edward Kmett. 2012. Lenses, Folds, and Traversals. In *Workshop on Generic Programming*.
- [23] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.
- [24] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
- [25] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.
- [26] redline6561. 2015. Immutable-structures: Common Lisp immutable data structures. <https://github.com/redline6561/immutable-structures>
- [27] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.
- [28] Erik Svedäng. 2018. Carp: A Statically Typed Lisp Without a GC. <https://github.com/carp-lang/Carp>
- [29] Mark Tarver. 2011. *The Shen Language*. Upaya Books.
- [30] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.
- [31] Sam Tobin-Hochstadt, Vincent St-Amour, Ryan Culpepper, Matthew Flatt, and Matthias Felleisen. 2011. Languages as Libraries. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. 132–141.
- [32] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *ACM SIGPLAN Notices*, Vol. 22. ACM, 227–242.